

DOCUMENT TECHNIQUE

v1.0

FLOWFORGE

Plateforme d'Automatisation ERP
avec Moteur de Workflow Visuel

Cahier des Charges — Spécification Fonctionnelle & Technique

Date : Août 2026

Statut : Approuvé — Phase de développement (Phase 1/8 réalisée)

Sommaire

1. Contexte & Objectifs
2. Périmètre du Produit
3. Exigences Fonctionnelles
4. Architecture Technique
5. Modèle de Données
6. Rôles, Permissions & Sécurité
7. Moteur d'Exécution de Workflow
8. API REST
9. Plan de Développement
10. Qualité & Tests
11. Environnement d'Exécution

1. Contexte & Objectifs

FlowForge est une plateforme SaaS de type **ERP (Enterprise Resource Planning)** orientée **automatisation de processus métier**. La solution permet aux entreprises de gérer leurs données opérationnelles (clients, produits, stocks, achats, factures) puis de **modéliser, exécuter et surveiller** des processus métier automatisés à travers un **constructeur de workflow visuel**.

1.1 Problématique

Les processus métier répétitifs (approbation d'une demande d'achat, relance de facture, alerte de stock) sont souvent gérés manuellement (e-mails, tableurs), ce qui engendre : lenteurs, pertes de traçabilité, erreurs humaines et absence de contrôle.

1.2 Objectifs

- Centraliser la gestion des données ERP au sein d'une application unique.
- Permettre la création **visuelle** de processus par des utilisateurs non-techniques (drag & drop).
- Exécuter automatiquement les processus déclenchés par des événements métier.
- Traçabilité complète : historique d'exécution, audits, statuts en temps réel.
- Assister la conception de workflow grâce à un **générateur IA**.

1.3 Public cible

Responsables opérationnels, gestionnaires, équipes financières et employés souhaitant automatiser des tâches quotidiennes sans écrire de code.

2. Périmètre du Produit

Le système se compose de **huit modules** cohérents :

N°	Module	Description
1	Simulation Mini-ERP	Gestion des utilisateurs/roles, clients, produits & inventaire, demandes d'achat, factures.
2	Gestion des Workflows	Création, édition, activation et versionnement de processus.
3	Constructeur Visuel	Interface React Flow : nœuds, connexions, configuration, sauvegarde/chargement.
4	Moteur d'Exécution	Runner, exécuteur de nœuds, évaluateur de conditions, actions, files d'attente.
5	Approbations & Notifications	Validation/refus, commentaires, e-mail et notifications internes.
6	Tableau de Bord	KPIs, graphiques de performance et latences.
7	Fonctionnalités avancées	Modèles, versionnement, journaux d'audit, suivi temps réel (Reverb).
8	IA	Générateur de workflow à partir d'un texte libre.

3. Exigences Fonctionnelles

3.1 Module 1 — Simulation Mini-ERP

Gestion des utilisateurs

- Authentification (enregistrement, connexion, déconnexion), tokens Sanctum.
- Quatre rôles : **Admin, Manager, Finance, Employee**.
- Assignation de rôles et permissions (RBAC).

Clients

CRUD (création, lecture, recherche, filtrage, mise à jour, suppression), statut actif/inactif.

Produits & Stock

- CRUD produits (nom, SKU unique, prix, quantité, seuil minimum).
- Mouvements de stock avec historique et référence morph.
- Détection automatique des **stocks faibles**.

Demandes d'achat

Créées par un employé ; statuts *pending / approved / rejected* ; montant, description.

Factures

Montant, échéance, statut *pending / paid / overdue* ; marquage payé ; détection auto des retards (scheduler).

Données de démonstration : 50 clients, 100 produits, 20 employés, 100 factures, 50 demandes d'achat générées par factory/seeder (données présentes après installation).

3.2 Module 2 — Gestion des Workflows

Un workflow associe des entités : nœuds (positionnés) et connexions. Entités versionnées et dotées de statut (*draft / active / paused / archived*).

3.3 Module 3 — Constructeur Visuel

Interface **React Flow** : glisser-déposer de nœuds, création de connexions, suppression, configuration via panneau latéral, sauvegarde et chargement. Types de nœuds :

- **Déclencheur (trigger)** : facture créée, demande d'achat créée, stock mis à jour.
- **Action** : envoyer un e-mail, notification, mettre à jour une entité, générer PDF.
- **Condition** : ex. `si invoice.amount > 5000`.
- **Approbation** : approbation manager / finance.
- **Délai** : attendre X jours.
- **Webhook** : appel HTTP externe.

3.4 Module 4 — Moteur d'Exécution

Démarrage automatique sur événement métier, exécution séquentielle des nœuds, évaluation des conditions, gestion des erreurs et sauvegarde de l'historique. Composants : `WorkflowRunner` , `NodeExecutor` , `ConditionEvaluator` , `ActionExecutor` . Appui aux jobs & files d'attente.

3.5 Module 5 — Approbations & Notifications

Demande d'approbation : *approuver / rejeter / commenter* ; historique ; notifications e-mail et internes.

3.6 Module 6 — Tableau de Bord

KPIs : workflows totaux/cours/complétés/échoués, temps moyen d'exécution, approbations en attente. Graphiques : performance, workflows les plus utilisés, latences processus.

3.7 Module 7 — Avancé

Modèles (approbation d'achat, relance invite, alerte stock, onboarding), versionnage des workflows, journaux d'audit (qui/quoi/quand), suivi temps réel des nœuds (running / completed / failed) via Reverb.

3.8 Module 8 — IA

Générateur de workflow : "*crée un workflow qui approuve les factures au-dessus de 10 000 TND*" → génération JSON (déclencheur + condition + approbation) importable dans le constructeur.

4. Architecture Technique

4.1 Vue d'ensemble des couches

Couche	Rôle	Technologies
Présentation	Interface SPA, constructeur visuel, dashboard	React · TypeScript · Vite · Tailwind · shadcn/ui · React Flow
API	Contrats JSON / REST, validation, ressources	Laravel Sanctum · FormRequests · API Resources
Application	Cas d'usage, services, orchestration	Services Layer · Actions
Domaine	Modèles métier = agrégats, événements de domaine	Eloquent (DDD concepts)
Infrastructure	Persistance, files, événements, notifications	Repositories · Jobs · Events · Notifications

4.2 Styles d'architecture

- **Monolithe modulaire** : le code est groupé par domaine (`Domain/Customer` , `Domain/Workflow` ...), favorisant la maintenance et un découpage ultérieur en microservices.
- **Principes SOLID** : responsabilité unique (services), inversion de dépendance (IPorts via interfaces de repository).
- **Couplage faible ERP ↔ Moteur** : les événements métier (ex. `PurchaseRequestCreated`) déclenchent des *listeners* qui résolut les workflows actifs puis jettent un *job*.
- **Exécution déterministe & reparable** : chaque nœud écrit une ligne `workflow_logs` , permettant de reprendre un run échoué.
- **Enumérations** pour les statuts (pas de chaînes magiques).

4.3 Structure des dossiers

```
workflow/  
  app/  
    Console/Commands/Demo/          # demo:reset  
    Domain/User/                    # utilisateurs, rôles, permissions, AuthService, RbacService  
    Domain/{Customer,Product,Invoice,PurchaseRequest,Approval,Workflow,Audit}/  
    Http/Controllers/Api/V1/        # contrôleurs API v1  
    Http/Requests/                  # FormRequests  
    Http/Resources/                 # API Resources  
    Http/Middleware/                # role / permission  
    Support/                        # ApiResponse  
  database/{migrations,factories,seeders}  
  routes/{api,channels,web}  
  tests/{Feature,Unit}  
  frontend/                         # future: React app
```

5. Conception de la Base de Données (MySQL)

5.1 Authentification & RBAC

Table	Colonnes clés
users	id, name, email(unique), department, job_title, password, remember_token
roles	id, name, slug(unique), description

Table	Colonnes clés
permissions	id, name, slug(unique), group
permission_role	role_id, permission_id (PF unique)
role_user	role_id, user_id (PF unique)
personal_access_tokens	Sanctum

5.2 Simulation ERP

Table	Colonnes clés
customers	company_name, contact_name, email, phone, status(enumeré)
products	name, sku(unique), price, stock_quantity, minimum_stock
stock_movements	product_id(FK), type, quantity, reference_type/reference_id (flou)
purchase_requests	employee_id(FK), amount, description, status(enumeré), approved_at
invoices	customer_id(FK), amount, due_date, status(enumer), paid_at

5.3 Workflow Engine

Table	Colonnes clés
workflows	name, description, status(enumer), version, created_by(FK), source_workflow_id(FK null)
workflow_nodes	workflow_id(FK), type(enumer), configuration(JSON), position_x, position_y
workflow_connections	workflow_id(FK), source_node, target_node, condition(JSON)
workflow_runs	workflow_id(FK), status(enumer), trigger_data(JSON), started_at, finished_at
workflow_logs	workflow_run_id(FK), node_id, status, message, metadata(JSON)
approvals	workflow_run_id(FK), node_id, approver_id(FK), status, comment, decided_at
approva_history	approval_id(FK), actor_id(FK), action, comment
audit_logs	actor_id(FK), action, entity_type, entity_id, before(JSON), after(JSON)
workflow_templates	name, description, definition(JSON), category

Note : index sur toutes les clés étrangères ; colonnes de statut modélisées en ENUM ; JSON pour les configurations de nœuds.

6. Rôles, Permissions & Sécurité

6.1 Matrice des rôles

Rôle	Permissions principales
Admin	Totalité de la matrice (users, customers, products, invoices, purchase_requests, workflows, approvals, dashboard, audit, AI)

Rôle	Permissions principales
Manager	View/create/update clients & produits, gère stock, approuve demandes d'achat & factures, gère workflows, approbations, dashboard, audit, AI
Finance	Vue clients & produits, approuve demandes d'achat, gère factures (mark_paid, approve), workflow exec, approbations, dashboard
Employee	Dashboard, vue produits, crée/lie demandes d'achat, vue factures, vue approbations

6.2 Sécurité

- Authentification par token **Sanctum** (Bearer).
- Mots de passe hachés (BCrypt 12 rounds).
- Middlewares `role:*` et `permission:*` sur les routes API.
- Validation systématique via FormRequests ; réponse d'erreur JSON normalisée.

7. Moteur d'Exécution de Workflow

7.1 Cycle de vie d'un workflow

Pour l'exemple cible "Approbation d'achat" :

```

Demande d'achat créée ? [Événement PurchaseRequestCreated]
  ? File d'attente (job RunWorkflow)
    ? WorkflowRunner : récupère le graphe, initialise un run
      ? NodeExecutor : exécute Déclencheur
        ? ConditionEvaluator : amount > 5000 ?
          ? ? Approbation Manager
            ? ? Approbation Finance
              ? ? Notification (e-mail)
                ? ? Action: mettre à jour le statut
          ? Logs de route + run finish
  
```

7.2 Composants clés

- `WorkflowRunner` : orchestration d'un run.
- `NodeExecutor` : dispatch par type de nœud.
- `ConditionEvaluator` : évalue les conditions (opérateurs, types).
- `ActionExecutor` : exécute les actions (mail, notification, update record, webhook, délai).
- **Approbation** : affirmation d'un run (`awaiting_approval`), reprise après décision.

8. API REST

Toutes les routes sous `/api/v1` , protégées par token. Enveloppe standardisée : `{ success, message, data, meta?, timestamps }` .

Méthode	Route	Protection	Description
POST	/auth/register	—	Inscription (rôle Employee)
POST	/auth/login	—	Connexion → token + user

Méthode	Route	Protection	Description
POST	/auth/logout	auth:sanctum	Révocation du token
GET	/auth/me	auth:sanctum	Utilisateur courant
GET	/users	permission:users.view	Liste paginée + recherche
PUT	/users/{user}/roles	permission:users.update	Assigner des rôles

Validation / erreurs : 401 (non authentifié), 403 (permissions insuffisantes), 422 (règles applicables), enveloppe `errors` .

9. Plan de Développement (Roadmap)

Phase	Livrable	Statut
0	Analyse & Conception (architecture, données, planning)	✓ Terminée
1	Backend Laravel : Sanctum, RBAC, API base	✓ Réalisée
2	Modules ERP (clients, produits/stock, demandes d'achat, factures) + démo	À faire
3	CRUD Workflow (nœuds, connexions, templates, versionnage, audit)	À faire
4	Constructeur React Flow (frontend)	À faire
5	Moteur d'exécution (Runner, Condition, Action) + événements/jobs	À faire
6	Approbations & notifications	À faire
7	Dashboard + heure temps réel (Reverb)	À faire
8	IA : générateur de workflow	À faire

10. Qualité & Tests

- Tests fonctionnels **Pest/PHPUnit** sur auth & RBAC (12 tests verts à ce stade).
- Tests sur base temporaires SQLite in-memory (ne touchent pas les données réelles).
- Suivi des fichiers : `php artisan test` , `demo:reset --fresh` .
- Journalisation (Log) pour traçabilité des erreurs.

11. Environnement d'Exécution

Composant	Version / Configuration
PHP	8.4 (herd-lite / XAMPP)
Laravel	12.x
Base de données	MySQL 8 (base <code>flowforge</code>)
File d'attente	<code>database</code> en dev (Redirigeable vers Redis en production)
Serveur dev	<code>php artisan serve --no-reload</code>
Comptes démo	admin@flowforge.dev · manager@flowforge.dev · finance@flowforge.dev · employee@flowforge.dev / mdp <code>password</code>

Document généré automatiquement — FlowForge ERP Automation Platform. Ce cahier des charges couvre les exigences, l'architecture et le planning ; il sera mis à jour à l'achèvement de chaque phase.